

POLITECNICO
MILANO 1863

Master's Degree in
Computer Science and Engineering

Formal Methods for Software Engineering – Project

Hop-paal

A Formal Frogger Journey

Academic Year: 2025/2026

Authors: **Nicolò Pallavicini** ([redacted])
Marco Puddu ([redacted])
Lorenzo Trenti ([redacted])
Stefano Ungaro ([redacted])

Instructors: **Pierluigi San Pietro**
Andrea Manini

Contents

Table of Contents **II**

List of Figures **III**

List of Tables **IV**

1 Introduction **1**
1.1 Game Assumptions and Specifications . . . 1
1.2 Document Structure . . . 1
1.3 Software and Hardware Specifications . . . 1

2 Model Description **3**
2.1 Map Abstraction and Entities . . . 3
2.2 Time Management and Atomicity . . . 3
2.3 Frogger's Automaton . . . 4
2.4 Stochastic Extensions . . . 5

3 Symbolic Verification **6**
3.1 Safety Properties . . . 6
3.2 Reachability Properties . . . 6
3.3 Verification . . . 7

4 Stochastic Analysis & Strategies **8**
4.1 Analysis of the 200-Point Milestone . . . 8
4.2 Quantitative Performance Metrics . . . 8
4.3 Results . . . 9
4.3.1 Analysis of Navigational Risk and Fatality Causes . . . 10
4.3.2 Analysis of Optimal Reward and Scoring Mechanics . . . 10

5 Conclusions **12**
5.1 Model Evaluation and Design Highlights . . . 12
5.2 Summary of Verification Insights . . . 12
5.3 Final Remarks . . . 12

References **13**

List of Figures

2.1 `GameEngine` automaton. 3
2.2 Frogger `Waiting` state. 4
2.3 Frogger `Playing` state. 4
2.4 Frogger `CheckState` state. 4
2.5 Frogger `Dead` state. 4
2.6 Truck's automaton with exponential rate $\lambda = 1/\text{speed}$. 5
2.7 New probabilistic branch point. 5

4.1 Comparative analysis of navigational risk for the *Baseline*, *Weights*, and *Smart* strategies. 10
4.2 Comparative analysis of progression depth for the *Baseline*, *Weights*, and *Smart* strategies. 10
4.3 Comparative analysis of optimal reward for the *Baseline*, *Weights*, and *Smart* strategies. 11
4.4 Comparative analysis of objective completion for the *Baseline*, *Weights*, and *Smart* strategies. 11
4.5 Comparative analysis of global survivability for the *Baseline*, *Weights*, and *Smart* strategies. 11

List of Tables

1.1 Software tools utilised in the project. 2
1.2 Hardware specifications of the machines used for formal verification. 2

3.1 Symbolic verification query results. 7

4.1 Stochastic verification 200-Point Milestone results. 8
4.2 Stochastic verification query results. ($\tau = 1000$ for *Baseline*/*Weights*, $\tau = 4000$ for *Smart*). 9

Chapter 1

Introduction

Originally developed by Konami and published by Sega in 1981, Frogger represents a milestone in the history of early video game development [1]. The primary objective of the game is to successfully guide the protagonist, Frogger, into five distinct home bays. Achieving this requires traversing a dense, horizontally scrolling highway populated by lethal vehicles, followed by a treacherous river filled with moving logs and turtles [1].

In this project, the focus is on formally modelling and verifying a simplified specification of the Atari 2600 version of Frogger [1], [2]. UPPAAL is a tool utilised for modelling, simulating and verifying real-time systems as networks of timed automata. Consequently, the game is modelled as a Network of Timed Automata (NTA) using the UPPAAL model checker. The purpose of this report is to document the critical design choices made during the modelling phase, verify the system's safety and liveness properties using TCTL formulae, and ultimately extend the model with stochastic behaviours to perform Statistical Model Checking (SMC).

1.1 Game Assumptions and Specifications

The model implements a single-player version of the game operating at difficulty level B, meaning Frogger may safely be carried off-screen by a floating object and wrap around to the opposite side of the map [1]. To ensure verifiable and realistic physics within the UPPAAL engine, the following core assumptions were established:

Time Scaling One time unit in the UPPAAL engine corresponds exactly to $0{,}1$ seconds of in-game time.

Zeno Behaviour Prevention To prevent unrealistic, infinite sequences of actions in zero time (Zeno behaviours), a strict delay of at least 1 time unit ($0{,}1\,\mathrm{s}$) is enforced between any two consecutive voluntary moves by Frogger.

Map Dimensions The game environment is defined as a fixed grid of 18×13 cells, featuring a safe median separating the vehicular highway from the river hazards.

1.2 Document Structure

The remainder of this report is structured as follows:

- **Chapter 2 (Model Description):** Details the architecture of the UPPAAL model, highlighting the abstraction of the map, the atomicity of the game engine, and the behavioural templates for Frogger and the hazards.
- **Chapter 3 (Symbolic Verification):** Presents the results of the exact model checking phase, detailing the queries used to verify the mandatory logical properties of the game.
- **Chapter 4 (Stochastic Analysis & Strategies):** Explores the Statistical Model Checking (SMC) extension, comparing the baseline random-walk movement probabilities against a custom, heuristic-based survival strategy.
- **Chapter 5 (Conclusions):** Summarises the findings of the verification processes and the effectiveness of the implemented movement policies.

1.3 Software and Hardware Specifications

The modelling, simulation, and verification processes were conducted using a specific set of software tools. The details of the software utilised for the development and documentation of this project are outlined in Table 1.1.

Software	Usage and Version
UPPAAL	Version 5.0.0. Utilised for the modelling, simulation, and formal verification of the Network of Timed Automata.
Overleaf	Used for LaTeX typesetting and for compiling this document.
Git/GitHub	Employed for version control and collaborative development tracking.

Table 1.1 – Software tools utilised in the project.

To ensure reproducibility and to document the computational resources required for the state-space exploration, all queries were executed on the hardware configurations detailed in Table 1.2.

Specification	Machine 1	Machine 2
CPU	Intel i7-11370H	AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx
RAM	16 GB	8 GB
OS	Fedora Linux 44 (Silverblue)	Windows 11 Home

Table 1.2 – Hardware specifications of the machines used for formal verification.

Chapter 2

Model Description

The formal translation of the Frogger game into a Network of Timed Automata (NTA) requires a careful balance between accurate simulation and state-space optimisation. This chapter details the architecture of the UPPAAL model, highlighting the structural design of the map, the centralised management of time, and the specific automata driving both the hazards and the player-controlled protagonist.

2.1 Map Abstraction and Entities

To mitigate state-space explosion, the 18×13 game grid is not modelled as an explicit two-dimensional array. Instead, an implicit geographic representation was chosen. The state of the map is entirely defined by the coordinate data of the entities occupying it.

To further restrict the exponential growth of the state space, individual hazards and platforms are purposely not modelled as separate Timed Automata. Instantiating dozens of concurrent automata for individual vehicles and logs would result in an intractable number of state interleavings. Therefore, a data-driven approach was adopted, treating these entities strictly as global variables whose states are centrally managed and updated by the `GameEngine` automaton presented later.

Hazards and platforms are structured using a custom `EntityConfig` blueprint, which defines static parameters such as size, base speed, and whether the entity possesses a diving mechanic (e.g., diving turtles) [1]. These blueprints are instantiated into arrays of `ActiveEntity` objects, which track dynamic variables including the current x and y coordinates and local movement timers. Specifically, the `ActiveEntity` struct encapsulates these coordinates alongside a `move_timer` and an `EntityConfig` reference, allowing global arrays (such as `vehicles` and `floats`) to be instantiated compactly and without redundancy.

Rather than each entity updating itself independently, the `GameEngine` iterates through these arrays to synchronously process their movements. Moreover, to replicate the continuous scrolling of the arcade game, this centralised movement logic implements Torus wrapping: a hidden buffer zone is defined via the global parameter `WRAP_SIZE`, currently set at 2 [2].

2.2 Time Management and Atomicity

In complex formal models, decoupling continuous time from discrete programmatic updates is vital to ensure verifiable accuracy [3]. This decoupling is managed by a centralised `GameEngine` template.

The engine alternates between two primary locations:

- **`Active` State:** A passive resting location constrained by the invariant $t \leq 1$. This forces the discrete simulation to wait exactly 1 time unit (0,1 s) before triggering the next physics tick [1].
- **`Evaluating` State:** Upon reaching 1 time unit, the engine transitions to this state, triggering the routine to advance all hazard arrays (`update_game()`). Crucially, `Evaluating` is marked as a *committed* location.

Using a committed location guarantees atomicity [4]. It ensures that immediately after the entities move, a comprehensive collision check (`is_deadly()`) is executed in zero time. If a lethal collision is detected at Frogger's exact coordinates, the engine synchronously fires an urgent broadcast on the `die` channel, eliminating the protagonist before the global clock is permitted to advance further.

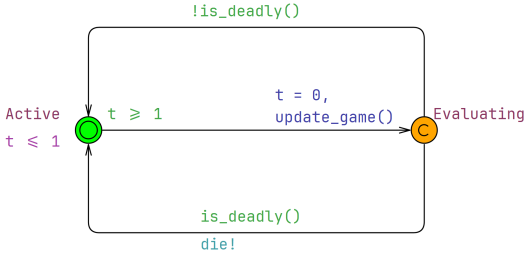


Figure 2.1 – `GameEngine` automaton.

2.3 Frogger's Automaton

The player-controlled agent is governed by the `Frogger` template, which tracks survival, scoring, and level progression through a series of discrete states:

- **`Waiting`:** Enforces the mandatory 2 s initial delay before movement is permitted, utilising a local clock invariant [1]. This is the landing state after a home bay is filled or after a death (if Frogger has spare lives).

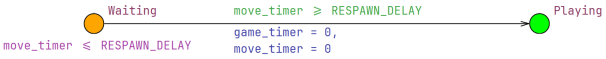


Figure 2.2 – Frogger `Waiting` state.

- **`Playing`:** The primary resting state between jumps, constrained by the global 30 s stage timeout invariant. Frogger can exit this state when its location becomes unsafe (by receiving the `die` signal), when time is up, or when a new move is performed. To prevent Zeno behaviour, at least 1 time unit must elapse between two consecutive moves (this is granted by guarding on the `move_timer`).

  Movement requests are evaluated by a dedicated validation function (`can_jump()`), which first ensures the jump complies with physical constraints, such as remaining within screen boundaries and avoiding previously filled home bays. In the simplified symbolic version, the jump is then performed by incrementing the y-coordinate. In the much complex stochastic version, the jump is performed differently (see Section 2.4 for more details).

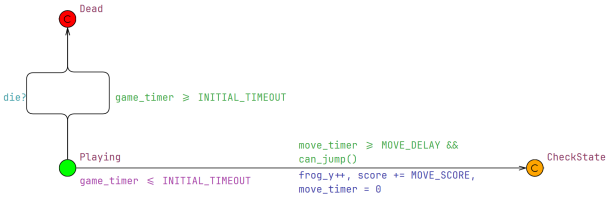


Figure 2.3 – Frogger `Playing` state.

- **`CheckState`:** A committed location reached immediately after a jump attempt. It freezes time to evaluate the new coordinates. If Frogger landed in an unsafe location, the system transitions to the `Dead` state. If the location is safe, it transitions back to the `Playing` state, and if a home bay was filled by the jump, it transitions to the `Scored` state, which checks whether all bays have been filled (transition to `GameWon` state) or not (transition back to `Waiting` state).

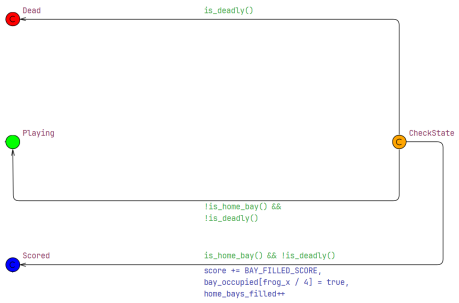


Figure 2.4 – Frogger `CheckState` state.

- **`Dead`:** A committed location reached after Frogger died. Here, the automaton checks for the number of spare lives and transitions back to the beginning of the map or to the `GameOver` state. This last state represents a sink location, since no more playing can be performed.

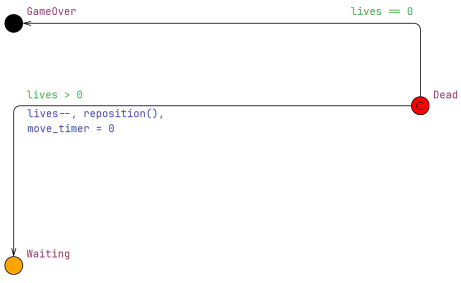


Figure 2.5 – Frogger `Dead` state.

2.4 Stochastic Extensions

To support Statistical Model Checking (SMC), specific deterministic elements of the baseline NTA were transformed to exhibit probabilistic behaviours.

Firstly, to model unpredictable traffic flow, the deterministic speed timers for truck entities were replaced with an exponential probability distribution. A dedicated `Truck` template was introduced, featuring a state with an exponential rate defined as $\lambda = 1/\text{speed}$ [2]. This ensures the trucks move continuously but at stochastically varied intervals. As a consequence, the `update_game()` function has been modified to skip the truck's standard array updating. By selectively isolating only the trucks into independent stochastic automata while keeping the remaining environment deterministically driven, the model achieves an optimal balance between probabilistic realism and computational tractability.

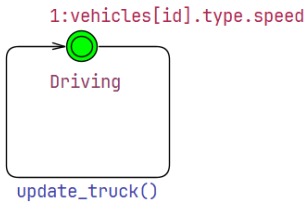


Figure 2.6 – Truck's automaton with exponential rate $\lambda = 1/\text{speed}$.

Secondly, Frogger's movement was converted into a probabilistic random walk [1]. The outgoing transition from the `Playing` state was modified into a probabilistic branch point. A central strategy selector assigns specific stochastic weights to the Up, Down, Left, and Right jumps. This architecture allows the UPPAAL verifier to simulate thousands of runs under different probabilistic movement policies. The implemented policies range from a purely random walk (*Baseline*, with directional weights distributed as 5-Up, 1-Down, 2-Left, 2-Right) to forward-biased (*Weights*, 8-Up, 0-Down, 1-Left, 1-Right) and heuristic-aware movement patterns (*Smart*). A comprehensive analysis of these specific strategies' impact on survival rates and scoring metrics is detailed in Chapter 4.



Figure 2.7 – New probabilistic branch point.

For the stochastic simulations, the basic jump validation logic presented before was replaced with a comprehensive `attempt_move()` function, which evaluates the chosen direction and executes the transition if physically valid. When the *Smart* strategy is active, this function introduces a heuristic look-ahead mechanism. As illustrated in Code 2.1, before committing to a spatial transition, the engine dynamically verifies if the target coordinates currently host a vehicle, lack a floating platform over the river, or coincide with the lethal walls separating the home bays.

```
// Heuristic look-ahead implementation
if (ACTIVE_STRATEGY == STRATEGY_SMART) {
    if (vehicle_at(new_x, new_y)) return;
    if (new_y >= 7 && new_y <= 11 && !platform_at(new_x, new_y)) return;
    if (new_y == MAX_Y && (new_x % 4 >= 2)) return;
}
```

Code 2.1 – The predictive safety guard preventing trivial mechanical deaths.

By performing this predictive assessment at the programmatic level rather than embedding it within a complex network of observer automata, the model actively prevents the agent from executing immediately suicidal moves while keeping the state-space strictly bounded. This architectural choice is fundamental to isolate the true efficacy of the algorithm's pathing decisions from random, unavoidable fatalities.

Chapter 3

Symbolic Verification

Formal verification is conducted by expressing desired system properties as Timed Computation Tree Logic (TCTL) formulae, which the UPPAAL engine evaluates exhaustively over the reachable state space [3]. Seven queries were defined and verified to evaluate the correctness of the model, with all queries returning a positive outcome (**satisfied**). Queries I and II target engine-level invariants and are verified using the `system GameEngine` in isolation, whilst Queries III to VII reason over global game states, necessitating the composition of the full `system Frogger, GameEngine`.

To maintain formal rigour, the following mathematical notation is employed throughout this chapter:

- V and F denote the sets of vehicles and floating objects (logs and turtles), respectively.
- y_a and x_a represent the y-coordinate and x-coordinate of entity a.
- Scored, Playing, and Dead represent the discrete locations of the `Frogger` automaton.
- score, lives, and t_{game} denote the game's global counters.
- $L_{\max}$ denotes the maximum number of lives.
- W denotes the world's width, defined as $X_{\max} + S_{\text{wrap}} + 1$. Note that, unlike the discrete implementation in UPPAAL, the modular arithmetic within TCTL formulae implicitly handles negative values, removing the requirement for coordinate correction.

3.1 Safety Properties

Safety properties assert that a bad condition never occurs across all reachable states.

Query I

$$\mathbf{A}\Box\,(\neg \texttt{deadlock})$$

This query verifies that the `GameEngine` never reaches a state in which no transition is enabled, thereby ensuring the absence of deadlocks within the engine's cycle. The invariant $t \leq 1$ on `Active` guarantees a tick fires at every time unit, and the `Evaluating` committed location always has two enabled outgoing edges (one per branch of `is_deadly()`), making stalling impossible.

Query II

$$\mathbf{A}\Box\left(\forall v \in V : y_v = y_v^{(0)} \wedge \forall f \in F : y_f = y_f^{(0)}\right)$$

This query verifies that every hazard entity remains confined to its designated row at all times. Since the function `update_game()` only ever modifies the x-coordinate of an entity, the initial y-assignment is never altered; the universal quantifier confirms this invariant holds in every reachable state.

3.2 Reachability Properties

Reachability properties ($\Diamond$) assert the existence of at least one execution trace leading to a target state.

Query III

$$\mathbf{E}\Diamond\left(\text{Scored} \;\wedge\; (x_{\text{frog}} = 0 \;\vee\; x_{\text{frog}} = 1) \;\wedge\; y_{\text{frog}} = 12\right)$$

This query verifies that Frogger can successfully reach and score the leftmost home bay at cells $(0, 12)$–$(1, 12)$. This is a basic liveness requirement confirming that at least one valid trajectory through the highway and river

sections exists for the far-left bay.

Query IV

$$\mathbf{E}\Diamond\left(\text{Scored} \ \wedge \ \text{score} = 17 \wedge \ (x_{\text{frog}} = 8 \ \vee \ x_{\text{frog}} = 9) \ \wedge \ y_{\text{frog}} = 12\right)$$

This query verifies that Frogger can reach the central bay at $(8, 12)$–$(9, 12)$ with a score of exactly 17 points. The constraint is not arbitrary: reaching row 12 from row 0 requires 12 upward jumps ($\times 1$ point each), plus 5 points for the bay, totalling $12 + 5 = 17$ only if Frogger took no backward steps, no lateral detours, and suffered no deaths — simultaneously validating spatial reachability and scoring correctness along an optimal path.

Query V

$$\mathbf{E}\Diamond\left(\text{Playing} \ \wedge \ y_{\text{frog}} = 6 \ \wedge \ \text{lives} = L_{\max} - 2\right)$$

This query verifies that a state exists where Frogger has lost 2 lives (reducing the `lives` counter by 2) and is positioned on the safe median strip at row 6. This confirms that the respawn and life-management logic operates correctly across multiple death-respawn cycles.

Query VI

$$\mathbf{E}\Diamond\left(\text{Playing} \ \wedge \ (y_{\text{frog}} = 8 \ \vee \ y_{\text{frog}} = 9 \ \vee \ y_{\text{frog}} = 11) \wedge \ \text{lives} = L_{\max} \ \wedge \ t_{\text{game}} = 100\right)$$

This query verifies that Frogger can stand on a log row exactly $10\,\text{s}$ into the stage with all lives intact. The joint constraint on `game_timer` and `lives` implicitly confirms that a valid platform existed at Frogger's coordinates at that instant, since any missing platform would have triggered `is_deadly()` and decremented `lives`.

Query VII

$$\mathbf{E}\Diamond\left(\text{Dead} \ \wedge \ \exists\, f \in F : \text{submerged}_f \ \wedge \ y_f = y_{\text{frog}} \wedge \ (x_{\text{frog}} - x_f) \bmod W < \text{size}_f\right)$$

This query verifies that Frogger can die as a direct result of a diving turtle submerging beneath him. Satisfaction requires three independent subsystems to interact correctly in sequence: the dive-cycle timer must advance into its `submerged` phase, Frogger must occupy the turtle's covered cells at that exact moment, and the `GameEngine` must detect the lethal condition and fire the `die!` broadcast — all within the same committed-location evaluation step.

3.3 Verification

Query	Result	Verification time (machine 1)	Verification time (machine 2)
I	satisfied	$\approx 0{,}141\,\text{s}$	$\approx 0{,}533\,\text{s}$
II	satisfied	$\approx 0{,}088\,\text{s}$	$\approx 0{,}282\,\text{s}$
III	satisfied	$\approx 0{,}034\,\text{s}$	$\approx 0{,}437\,\text{s}$
IV	satisfied	$\approx 0{,}009\,\text{s}$	$\approx 0{,}082\,\text{s}$
V	satisfied	$\approx 0{,}136\,\text{s}$	$\approx 0{,}401\,\text{s}$
VI	satisfied	$\approx 6{,}714\,\text{s}$	$\approx 17{,}895\,\text{s}$
VII	satisfied	$\approx 14{,}893\,\text{s}$	$\approx 46{,}489\,\text{s}$

Table 3.1 – Symbolic verification query results.

Chapter 4

Stochastic Analysis & Strategies

4.1 Analysis of the 200-Point Milestone

First, the target query has been analysed: completing the first level with a final score of 200 points or more. Under these conditions, the *Baseline* strategy yields a success probability of less than 5% (Table 4.1). This low success rate stems from the game mechanics and the inherent difficulty of accumulating the required points. Specifically, a strictly forward-moving strategy, where the agent only moves upward to reach the home bases, caps the maximum score at 185 points. Therefore, to achieve 200 points, the frog must intentionally backtrack (move down) to accumulate extra points from subsequent upward movements. However, executing these backward movements significantly increases the probability of failing.

Applying the alternative strategies, *Weights* and *Smart*, does not drastically alter this specific outcome. The success probability remains exceptionally low (3,1% for the *Smart* agent) due to the extreme difficulty of the objective. Furthermore, in these specific strategies, the weight assigned to downward movement is set to zero. This configuration effectively prevents the agent from executing the necessary backtracking, making it practically impossible to accumulate the 200 points required strictly within the confines of the first level.

Query	STRATEGY BASELINE	STRATEGY WEIGHTS	STRATEGY SMART
I	$\leq 0{,}049\,944\,1$ (95% CI)	$\leq 0{,}049\,944\,1$ (95% CI)	$0{,}030\,651\,4 \pm 0{,}030\,366\,9$ (95% CI)

Note: All computational estimations were performed using the hardware configuration of Machine 1.

Table 4.1 – Stochastic verification 200-Point Milestone results.

4.2 Quantitative Performance Metrics

To facilitate a comprehensive comparison between the *Baseline*, *Weights*, and *Smart* strategies, a suite of probabilistic and expected-value queries has been formulated. To allow the advanced *Smart* strategy sufficient time to exhibit its long-term progression across multiple levels, a variable simulation bound τ was introduced. Specifically, τ is set to 1000 time units for the *Baseline* and *Weights* strategies, and extended to 4000 time units for the *Smart* strategy.

These metrics are designed to evaluate the trade-offs between progression, risk-taking, and survival:

I 200-**Point Milestone:** $\Pr[\leq \tau](\Diamond(\text{level} = 1 \wedge \text{Scored} \wedge \text{bays}_{\text{filled}} = 5 \wedge \text{score} \geq 200 - S_{\text{bonus}}))$
Calculates the probability that the agent successfully completes the first level while accumulating a final score of at least 200 points. To circumvent the atomicity of the level transition, the query intercepts the committed `Scored` location exactly when the fifth bay is filled on level 1. At this precise execution step, achieving a base score of ≥ 100 logically guarantees a total score of ≥ 200 upon the application of the 100-point completion bonus (S_{bonus}).

II **Optimal Reward:** $\mathbb{E}[\leq \tau; 1000](\max : \text{score})$
Evaluates the expected maximum score achieved within the time bound to determine the reward maximisation potential of each strategy.

III **Collision Risk:** $\mathbb{E}[\leq \tau; 1000](\max : \text{stat}_{\text{vehicle}})$
Measures the frequency of collisions with vehicles, serving as a metric for the agent's safety during the road-crossing phase.

IV **Navigational Risk:** $\mathbb{E}[\leq \tau; 1000](\max : \text{stat}_{\text{drowned}})$
Assesses the probability of failure within the river section by calculating the expected maximum drowning

events.

V Temporal Efficiency: $\mathbb{E}[\leq \tau; 1000](\max : \text{stat}_{\text{timeout}})$
Quantifies the frequency of level timeouts, indicating whether a strategy is too conservative or lacks time-management efficiency.

VI Reachability Probability: $\Pr[\leq \tau](\Diamond y_{\text{frog}} \geq 7)$
Determines the probability that the agent successfully traverses the initial hazard zones and reaches the upper half of the map.

VII Global Survivability: $\mathbb{E}[\leq \tau; 1000](\max : \text{stat}_{\text{total_lifespan}})$
Computes the expected overall duration of a full game, providing an aggregate measure of the strategy's ability to prolong the playthrough.

VIII Progression Depth: $\mathbb{E}[\leq \tau; 1000](\max : y_{\text{frog}})$
Tracks the maximum forward distance reached by the frog, illustrating the average depth of progression.

IX Objective Completion: $\mathbb{E}[\leq \tau; 1000](\max : \text{bays}_{\text{filled}})$
Directly measures the expected number of goal states reached per level, which serves as the primary indicator of mission success.

X Level Progression: $\mathbb{E}[\leq \tau; 1000](\max : \text{level})$
Tracks the maximum level reached, evaluating the agent's capability to sustain operational success across increasingly difficult game stages.

XI Average Lifespan per Life: $\mathbb{E}[\leq \tau; 1000](\max : \text{stat}_{\text{lifespan_to_die}})$
Computes the average time elapsed before the agent loses a single life, isolating the per-life survival duration from the global lifespan.

4.3 Results

The following section presents the quantitative outcomes of the stochastic model checking queries defined previously. Table 4.2 summarises the statistical estimates derived from the simulation runs across the three proposed agent strategies. A comparative evaluation of these findings reveals significant differences in the agent's navigational efficiency, primary causes of termination, and overall success in the environment.

Query	STRATEGY BASELINE	STRATEGY WEIGHTS	STRATEGY SMART
I	$\leq 0{,}049\,944\,1$ (95% CI)	$\leq 0{,}049\,944\,1$ (95% CI)	$0{,}030\,651\,4 \pm 0{,}030\,366\,9$ (95% CI)
II	$21{,}492 \pm 0{,}413\,036$ (95% CI)	$21{,}325 \pm 0{,}337\,403$ (95% CI)	$615{,}781 \pm 21{,}0819$ (95% CI)
III	$3{,}986 \pm 0{,}057\,569\,4$ (95% CI)	$3{,}582 \pm 0{,}059\,397\,5$ (95% CI)	$4{,}898 \pm 0{,}028\,553\,7$ (95% CI)
IV	$1{,}059 \pm 0{,}068\,306\,9$ (95% CI)	$1{,}506 \pm 0{,}062\,886\,6$ (95% CI)	$0{,}108 \pm 0{,}020\,054\,4$ (95% CI)
V	≈ 0	≈ 0	$0{,}007 \pm 0{,}005\,176\,25$ (95% CI)
VI	$0{,}632\,313 \pm 0{,}048\,956\,9$ (95% CI)	$0{,}829\,588 \pm 0{,}046\,170\,9$ (95% CI)	$\geq 0{,}950\,056$ (95% CI)
VII	$75{,}945 \pm 1{,}670\,58$ (95% CI)	$47{,}286 \pm 0{,}771\,014$ (95% CI)	$1195{,}63 \pm 34{,}3433$ (95% CI)
VIII	$6{,}565 \pm 0{,}118\,611$ (95% CI)	$7{,}468 \pm 0{,}102\,601$ (95% CI)	$11{,}975 \pm 0{,}028\,341$ (95% CI)
IX	$0{,}004 \pm 0{,}003\,918\,79$ (95% CI)	$0{,}005 \pm 0{,}004\,379\,14$ (95% CI)	$4{,}939 \pm 0{,}022\,323\,2$ (95% CI)
X	≈ 1	≈ 1	$4{,}282 \pm 0{,}112\,839$ (95% CI)
XI	$16{,}132 \pm 0{,}372\,797$ (95% CI)	$9{,}482 \pm 0{,}142\,414$ (95% CI)	$14{,}72 \pm 0{,}615\,932$ (95% CI)

Note: All computational estimations were performed using the hardware configuration of Machine 1.

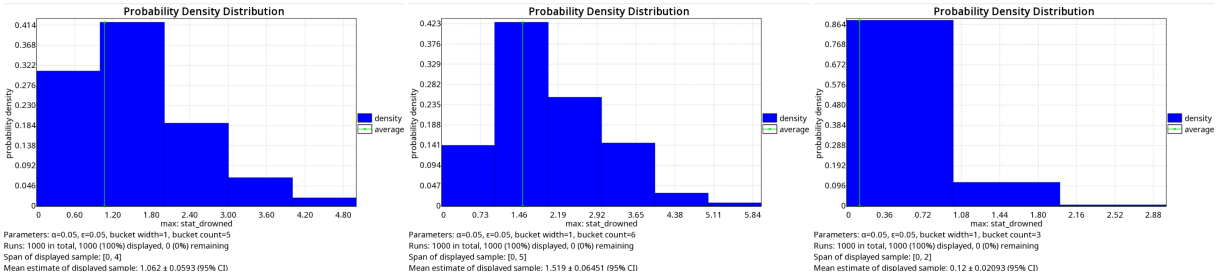
Table 4.2 – Stochastic verification query results. ($\tau = 1000$ for *Baseline*/*Weights*, $\tau = 4000$ for *Smart*).

4.3.1 Analysis of Navigational Risk and Fatality Causes

An analysis of the fatality metrics (Figure 4.1) reveals a profound shift in risk distribution across the strategies. The *Baseline* strategy yields an expected maximum of 1,059 drowning events, whereas the forward-biased *Weights* strategy shows an increased river-based risk profile, averaging 1,506 drownings. As illustrated in Figure 4.2, this increase occurs because the *Weights* agent reaches the hazardous river section more frequently (average y-coordinate of 7,468, Figure 4.2b).

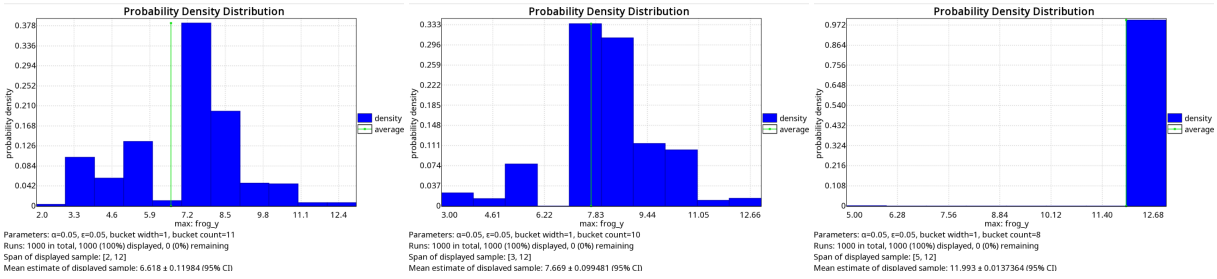
Conversely, the *Smart* strategy drastically reduces navigational risk in the water, recording a near-zero expected drowning statistic (0,108, Figure 4.1c). This confirms that the heuristic logic is structurally infallible when evaluating platform safety. However, this perfection in water navigation causes a stark displacement of fatalities: the *Smart* agent perishes almost exclusively by being hit by vehicles (4,898 times). This phenomenon is a direct consequence of its hazard-aware nature. To avoid being hit in the following row, the agent frequently waits on the highway for an optimal configuration, inadvertently exposing itself to prolonged periods within the dense, continuous traffic of the current row.

Furthermore, this cautious, waiting-based approach is responsible for a measurable, albeit slight, emergence of timeout fatalities (0,007), which are completely absent in the more reckless baseline configurations. Crucially, summing the mean fatalities for the *Smart* strategy (4,898 vehicles +0,108 drowned +0,007 timeout) yields 5,013. This mathematically corroborates that the model flawlessly accounts for the 5 total lives allotted to the player per game, validating the absolute structural integrity of the automaton.



(a) Navigational Risk *Baseline* **(b)** Navigational Risk *Weights* **(c)** Navigational Risk *Smart*

Figure 4.1 – Comparative analysis of navigational risk for the *Baseline*, *Weights*, and *Smart* strategies.



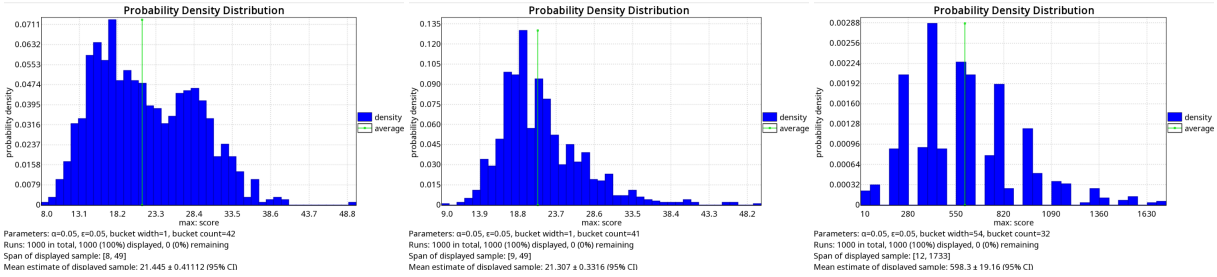
(a) Progression Depth *Baseline* **(b)** Progression Depth *Weights* **(c)** Progression Depth *Smart*

Figure 4.2 – Comparative analysis of progression depth for the *Baseline*, *Weights*, and *Smart* strategies.

4.3.2 Analysis of Optimal Reward and Scoring Mechanics

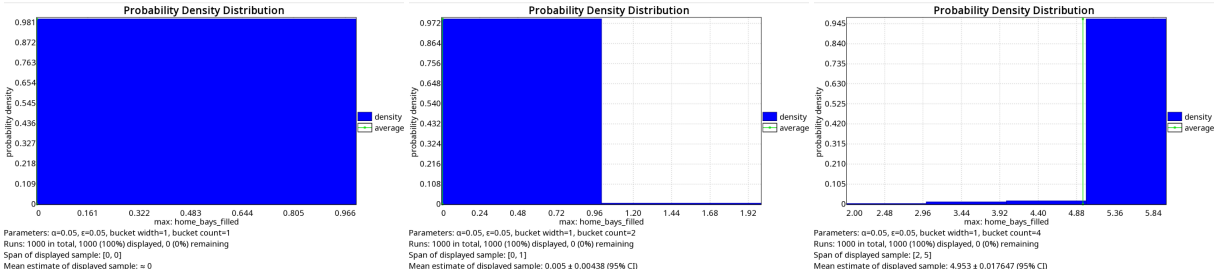
As illustrated in Figure 4.3, the evaluation of the expected maximum score reveals a staggering performance contrast between the standard approaches and the optimised *Smart* configuration. The *Baseline* and *Weights* strategies yield statistically indistinguishable and remarkably low mean scores of 21,492 and 21,325, respectively. Notably, the *Weights* strategy's forced forward bias completely fails to translate into better performance: while it marginally increases the progression depth, it forces the agent into river hazards prematurely, nearly halving its average lifespan per life (from 16,132 to 9,482) without generating additional points. Cross-referencing these scores with the objective completion data in Figures 4.4a and 4.4b, it is evident that neither strategy successfully

fills any home bays (averaging ≈ 0). Consequently, their entire score is derived purely from progressing upward approximately 21 times before the agent inevitably perishes, rarely advancing past the first level (Query X ≈ 1). This remarkably low survivability is further corroborated by Figures 4.5a and 4.5b (global lifespans of 76 and 47 steps, respectively).



(a) Optimal Reward *Baseline* **(b)** Optimal Reward *Weights* **(c)** Optimal Reward *Smart*

Figure 4.3 – Comparative analysis of optimal reward for the *Baseline*, *Weights*, and *Smart* strategies.

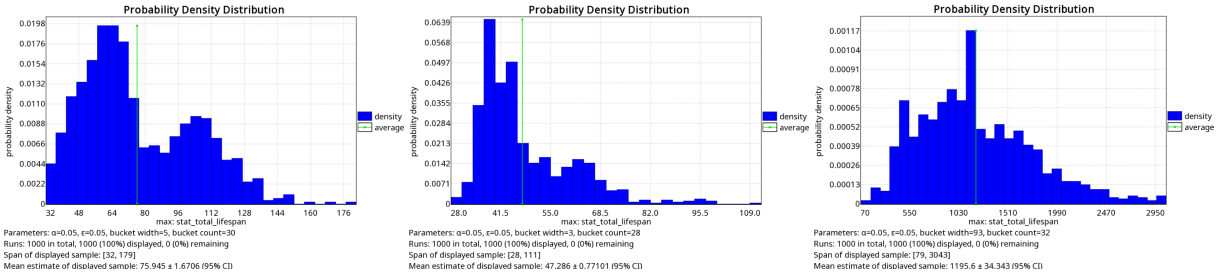


(a) Objective Completion *Baseline* **(b)** Objective Completion *Weights* **(c)** Objective Completion *Smart*

Figure 4.4 – Comparative analysis of objective completion for the *Baseline*, *Weights*, and *Smart* strategies.

On the other hand, the *Smart* strategy demonstrates a monumental performance leap, achieving an expected mean score of 615,781 (Figure 4.3c). The inclusion of the per-life survival metric (`stat_lifespan_to_die`, Query XI) provides crucial context for this success. The *Smart* agent's average time-to-die (14,72) is comparable to the *Baseline* (16,132), indicating that when a fatality does occur, it is often swiftly caused by the dense highway traffic while the agent waits for a safe path. However, its overall game lifespan (`stat_total_lifespan`) skyrockets to an impressive 1195,63 steps.

This immense disparity confirms the agent's profound pathing efficiency: by rapidly and systematically filling home bays (averaging 4,939 per simulation) without aimlessly lingering, it repeatedly clears levels and resets the game state, massively prolonging the overall playthrough. Furthermore, the progression metrics (Query X) reveal that the agent consistently reaches an expected maximum level of 4,282, proving its capability to sustain operational success even as the global timeout constraints become progressively stricter in advanced stages. This definitively establishes the heuristic approach as a highly efficient, game-solving strategy.



(a) Global Survivability *Baseline* **(b)** Global Survivability *Weights* **(c)** Global Survivability *Smart*

Figure 4.5 – Comparative analysis of global survivability for the *Baseline*, *Weights*, and *Smart* strategies.

Chapter 5

Conclusions

This project successfully demonstrated the formal modelling, symbolic verification and stochastic analysis of an adapted Atari 2600 version of *Frogger* using UPPAAL model checker. By translating complex real-time game mechanics into a structured Network of Timed Automata (NTA), the boundaries between exact state-space exploration and statistical performance were thoroughly explored.

5.1 Model Evaluation and Design Highlights

A primary challenge of this implementation was mitigating the state-space explosion inherently tied to tracking dozens of moving entities. In this project these problems were bypassed through key architectural decisions:

- **Implicit Map Representation**: Forgoing a literal two-dimensional array in favour of a data-driven, coordinate-tracking approach via `ActiveEntity` arrays drastically optimised RAM utilisation during verification.
- **Centralised Time and Atomicity**: Incorporating a single `GameEngine` to synchronously orchestrate physics ticks and immediate collision checking via committed states eliminated intractable interleaving. This ensured that safety evaluations occurred in zero time before the global clock advanced.

5.2 Summary of Verification Insights

The dual-phase evaluation yielded robust insights into the game's underlying logic and the efficacy of algorithmic decision-making.

- **Symbolic Exhaustiveness**: All seven TCTL queries were satisfied, as shown in Table 3.1. This confirmed basic structural invariants and proved that complex, multi-layered paths were reachable within the system's physical constraints.
- **The** 200**-Point Milestone Paradox**: SMC analysis highlighted that achieving a score ≥ 200 points within the first level is statistically negligible ($< 5\%$) for baseline random walks and structurally impossible for pure forward-biased models, as shown in Table 4.1. Because forward movement alone caps the score at 185, this threshold mathematically demands risky backtracking, a behaviour deliberately restricted by optimised survival heuristics.
- **The Strategic Shift**: The introduction of the *Smart* strategy resulted in a monumental performance leap, elevating the expected maximum score from a meagre ≈ 21 points to an impressive $615{,}781$, query II in Table 4.2. While the *Baseline* and *Weights* configurations routinely failed to secure a single home bay, the *Smart* strategy enabled the agent to successfully clear the first level in nearly every run, sustaining operational progression up to an expected maximum level of $4{,}282$, query X in Table 4.2.

5.3 Final Remarks

Interestingly, the quantitative results highlighted a compelling behavioural trade-off: by rendering the frog virtually immune to drowning ($0{,}108$ expected events, query IV in Table 4.2), the *Smart* heuristic forced the agent to wait on the highway for optimal configurations, inadvertently displacing its primary cause of death entirely to vehicle collisions ($4{,}898$, query III in Table 4.2). This displacement of risk reveals a nuanced structural phenomenon within the stochastic environment. While the forward-biased random walks of the *Baseline* and *Weights* configurations exhibit high recklessness by frequently drowning in the river section due to a blind lack of spatial awareness, the *Smart* strategy's defensive local look-ahead guards essentially transfer the operational risk backward. Because the agent refuses to make suicidal jumps into unsafe rows, it remains gridlocked within the dense, continuous traffic of the lower highway lanes, drastically increasing its temporal exposure to lethal vehicle arrays.

References

[1] Parker Brothers, *Frogger: Instruction manual*, For Atari 2600 & Sears Video Game Systems. Under license from Sega Enterprises, Inc., 1982. [Online]. Available: https://www.gamesdatabase.org/Media/SYSTEM/Atari_2600//Manual/formated/Frogger_-_1983_-_Starpath_Corporation.pdf.

[2] Andrea Manini, Pierluigi San Pietro, *Hop-paal: A formal frogger journey*, https://webeep.polimi.it.

[3] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on uppaal," in *Formal Methods for the Design of Real-Time Systems*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 200–236. DOI: 10.1007/978-3-540-30080-9_7.

[4] UPPAAL developers, *Uppaal documentation*, https://docs.uppaal.org, Accessed: 2026-05-04.